

cvc::nav + GRL-SNAM — Quickstart

The one-page command list to **see the Austin navigation demos running**. Two routes: **A. run the published demos off cvcpkg** (nothing to build), or **B. build everything yourself**. For the *why* — cvc::nav internals, how it maps to GRL-SNAM, the training loop, the `.cvcnav` format — see the full guide: [cvc-nav-and-grl-snam-guide.md](#).

0. Install cvcpkg (once)

```
curl -fsSL https://cvcpkg.org/install.sh | sh          # macOS / Linux
```

```
irm https://cvcpkg.org/install.ps1 | iex              # Windows PowerShell
```

(`pip install cvcpkg` also works.) Open a new shell so `cvcpkg` is on `PATH`.

Platforms: Linux (x86-64) is fully supported. **Windows (x86-64) works** — the Python stack (`pycvc-gl-cp311`, `torch-cp311`, ...) and the native `cvcgl-examples` demos publish for Windows; use `.\env\Scripts\activate`. **macOS is coming** (blocked on a VTK macOS bundle on the catalog). The wasm demos run in any browser.

A. Run the published demos (no build)

Everything is prebuilt on cvcpkg — the bindings, the pretrained model, the Austin geometry. Nothing to compile.

```
# 1. one prefix – the demos, the pretrained model, AND the Austin scene all come
#    along: grl-snam-cp311 pulls grl-snam-weights + scene-austin-south itself.
cvcpkg install grl-snam-cp311 --prefix ./nav-env

# 2. activate it (puts `grl-snam` and the bindings on PATH)
. ./nav-env/bin/activate          # Windows: .\nav-env\Scripts\activate

# 3. run a demo – opens its own window, no VolRover3 (the default)
grl-snam demo --list
grl-snam demo austin-freedrive
```

- Swap `cp311` → `cp312` / `cp313` to match your Python interpreter (the column must match — mixed columns won't import).
- `grl-snam demo NAME` runs **standalone** — a live `pycvc_gl` window that drives the chase camera + metrics itself (needs a display). Pass `--volrover3` to run the same demo *inside* VolRover3 instead (`grl-snam demo austin-freedrive --volrover3`), which needs the `volrover3` app on `PATH`.
- The install pulls the full ~110 MB `scene-austin-south` by default. Want the ~10 MB bundle instead? `cvcpkg install scene-austin-south-small` into the same prefix (same layout, same path) — it wins as the more-specific install.

The registered demos:

name	what it shows
austin-freedrive	learned SDF policy finds its OWN way A→B across Austin (no route)
austin-learned	A* route spine + learned SDF local control (GRL-SNAM stagewise)
austin-patrol	grounded A* street patrol (non-learned baseline)
austin-planner	surrogate planner in its native sparse-obstacle regime
lab	analytic terrain + an agent walking a draped loop

Where the model and the Austin geometry actually live

A frequent snag: the `scene-austin-south` **recipe** in `graphics-assets` is only a *package definition* (source: type: none) — there is **no scene data in that git repo, by design**. The real bytes are the **published cvcpkg package**; you get them with `cvcpkg install`, never from the recipe repo.

Everything below is public — no token, no login:

package	installs to	contents
grl-snam-weights	<prefix>/share/grl-snam-weights/	coef_sdf.pt (torch CoefMLP) + coef_sdf.cvcnav (native)
scene-austin-south	<prefix>/share/cvc-scenes/austin_south/	full-res terrain + buildings + imagery (~110 MB)
scene-austin-south-small	(same path)	~10 MB downscaled (same file layout)
scene-austin-south-web	(same path)	~10 MB, tuned for the wasm/browser build

Point the demos at your own model or a bundle elsewhere without reinstalling:

```
export GRL_SNAM_CHECKPOINT=/path/to/coef_sdf.pt
export GRL_SNAM_SCENE_BUNDLE=/path/to/austin_south
```

Sanity-check what landed:

```
ls ./nav-env/share/cvc-scenes/austin_south      # terrain.json buildings.glb
satellite.png ...
ls ./nav-env/share/grl-snam-weights             # coef_sdf.pt  coef_sdf.cvcnav
```

B. Build everything yourself

GRL-SNAM from source (Python)

`sdf_nav` and the demos ship *inside* GRL-SNAM — the only build-side pieces from `cvcpkg` are the bindings + torch.

```
git clone https://github.com/CVC-Lab/GRL-SNAM && cd GRL-SNAM
cvcpkg install pycvc-gl-cp311 torch-cp311 --prefix ./env # bindings + torch
. ./env/bin/activate
pip install -e . # grl-snam itself (editable)
cvcpkg install grl-snam-weights scene-austin-south --prefix ./env # data (a source
                                                                    # checkout doesn't pull the
                                                                    # recipe's deps)
grl-snam demo austin-freedrive
```

Train your own coefficients instead of the published ones:

```
grl-snam build-sdf ./env/share/cvc-scenes/austin_south # -> nav_sdf.npz
grl-snam train nav_sdf.npz -o checkpoints/coef_sdf.pt # --steps 1500 --seed 0
GRL_SNAM_CHECKPOINT=checkpoints/coef_sdf.pt grl-snam demo austin-freedrive
```

Native C++ nav demos (no Python)

The pure-C++ demos (`nav_city_swarm`, `nav_fog_ghost`, `nav_finale`, `nav_city_drive`, `lsystem_forest`, ...) are the `cvcgl-examples` `cvcpkg` package — **no Python, no torch, no pycvc**. Just the C++ GL stack (`libcvc` + `VTK` + `boost` + `imgui`). Three ways in:

Run the prebuilt binaries (no build):

```
cvcpkg install cvcgl-examples --prefix ./demos
./demos/bin/nav_city_swarm # reactive cvc::nav swarm on a synthetic city
./demos/bin/nav_fog_ghost # fog-of-war "ghost" story (top-down map)
./demos/bin/nav_finale # 2-act pursuit; auto-finds the Austin bundle if present
./demos/bin/lsystem_forest # procedural island
```

Every demo runs with zero args (`RPATH` is baked in — no `LD_LIBRARY_PATH`). To drive the *real* Austin scene + trained policy, install the data and point the swarm/drive demos at it:

```
cvcpkg install scene-austin-south grl-snam-weights --prefix ./demos
CVC_NAV_BUNDLE=./demos/share/cvc-scenes/austin_south \
CVC_NAV_WEIGHTS=./demos/share/grl-snam-weights/coef_sdf.cvcnav \
./demos/bin/nav_city_swarm # (or --bundle DIR / --agents N / --belief
grouped ...)
```

Build them via the recipe (the easy build):

```
git clone https://github.com/transfix/libcvc && cd libcvc
cvcpkg build cvcgl-examples --prefix ./demos # resolves the C++ deps + builds +
installs to ./demos/bin
```

Build them directly with CMake:

```
git clone https://github.com/transfix/libcvc && cd libcvc
# C++ GL deps only – libcvc + VTK + boost + imgui (NO python/pycvc/numpy/swig):
cvcpkg install-deps cvcpkg/recipes/cvcgl-examples --prefix ./deps --config release
cmake -G Ninja -S src/cvcGL -B build \
  -DCMAKE_BUILD_TYPE=Release -DBUILD_SHARED_LIBS=OFF \
  -DCVC_BUILD_EXAMPLES=ON -DCGAL_Boost_USE_STATIC_LIBS=OFF \
  -DCMAKE_PREFIX_PATH="$PWD/deps"
cmake --build build -j --target nav_city_swarm # or nav_city_drive / nav_fog_ghost /
nav_finale / lsystem_forest
./build/examples/nav_city_swarm
```

-DBUILD_SHARED_LIBS=OFF static-links cvcGL into each binary; -DCGAL_Boost_USE_STATIC_LIBS=OFF is required so find_package(Boost) accepts cvcpkg's shared Boost. Targets: nav_city_swarm, nav_city_drive, nav_fog_ghost, nav_finale, lsystem_forest, terrain_lab, bunny_shadow, ...

wasm (browser)

Prebuilt, hosted: the full gallery is live at <https://transfix.github.io/libcvc/> — e.g. /nav_city_swarm/, /nav_city_drive/, /nav_fog_ghost/.

Prebuilt, run locally: the cvcpkg wasm package ships the lsystem_forest demo plus a launcher that serves it cross-origin-isolated:

```
cvcpkg install cvcgl-examples --platform wasm --arch wasm32 --link static --prefix ./
demos
./demos/bin/cvcgl-examples-web # serves http://localhost:8811 and opens a
browser
```

Build the full wasm gallery from source (this one includes the nav demos):

```
git clone https://github.com/transfix/libcvc && cd libcvc
# 1. Emscripten SDK + wasm C++ deps (a RENDERING-enabled VTK >= 9.5.0+cvc.3):
cvcpkg install emsdk --platform linux --prefix /opt/cvc-wasm/emsdk
export CVC EMSDK_DIR=/opt/cvc-wasm/emsdk
cvcpkg install boost zstd --platform wasm --arch wasm32 --link static --prefix /opt/cvc-
wasm/deps
cvcpkg build vtk --platform wasm --local --prefix /opt/cvc-wasm/deps
export CVC_WASM_DEPS=/opt/cvc-wasm/deps
# 2. build + serve the gallery:
./src/cvcGL/examples/wasm/build-wasm-demo.sh # -> build-wasm/gallery/
python3 -m http.server -d build-wasm/gallery 8811 # open http://localhost:8811
# threaded variant: build-wasm-demo.sh --pthread, then serve with
# src/cvcGL/examples/wasm/serve.py (it sends the COOP/COEP headers threads need).
```

Preload the Austin scene into the wasm build with -DCVC_WASM_BUNDLE=<austin_dir> / -DCVC_WASM_NAV_WEIGHTS=<coef_sdf.cvcnav> — see the full guide.

Gotchas

- “the recipe has no scene data” — correct, it's data-free by design; install the *package* (table above) and the data lands under share/.

- **grl-snam demo NAME needs a display** (it's standalone by default, from `grl-snam-cp31X ≥ 0.2.1+cvc.4` ; older revisions defaulted to VolRover3 and took `--standalone`). Headless / no display? Render offscreen from Python (`pycvc_gl.SceneRenderer(scene, w, h, offscreen=True).writePNG("frame.png")`), or run a native `cvcgl-examples demo (cvcpkg install cvcgl-examples)`).
 - **column must match your interpreter** — `-cp311 / -cp312 / -cp313` .
 - **want the VolRover3 window instead?** — `grl-snam demo <name> --volrover3` shells out to `volrover3 --run-job` ; set `VOLROVER3_BIN` if `volrover3` isn't on `PATH` .
-

Details on `cvc::nav`, the GRL-SNAM mapping, and training: [cvc-nav-and-grl-snam-guide.md](#).